

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №4
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Алгоритм роя частиц.

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение алгоритма роя частиц для решения задачи поиска минимума функции.

Алгоритм.

Пусть требуется найти минимум целевой функции:

$$f(\mathbf{x}) \rightarrow \min, \mathbf{x} \in D \subset \mathbb{R}^n$$

где:

- n — размерность пространства поиска;
- D — допустимая область, задаваемая ограничениями:
- $D = \{\mathbf{x} \mid x_{\min}^i \leq x_i \leq x_{\max}^i, i = 1, \dots, n\}$

В алгоритме используется множество частиц $\mathcal{P} = \{P_1, P_2, \dots, P_m\}$, где m — размер роя.

Каждая частица P_j в момент времени tt характеризуется:

- положением $\mathbf{x}_j(t) \in \mathbb{R}^n$;
- скоростью $\mathbf{v}_j(t) \in \mathbb{R}^n$;
- лучшим найденным ею положением $\mathbf{p}_j$ (личный опыт);
- значением целевой функции в этом положении $f(\mathbf{p}_j)$.

Глобально лучшим положением среди всех частиц является:

$$\mathbf{g} = \arg \min_{\mathbf{p}_j} f(\mathbf{p}_j), \quad j = 1, \dots, m$$

В канонической реализации алгоритма роя частиц используется нормировка коэффициентов, которая позволяет сделать алгоритм менее чувствительным к выбору параметров и обеспечивает теоретическую сходимость.

Скорость частицы обновляется по следующему правилу:

$$\mathbf{v}_j(t+1) = \chi [\mathbf{v}_j(t) + \varphi_r \cdot r_p \cdot (\mathbf{p}_j - \mathbf{x}_j(t)) + \varphi_s \cdot r_g \cdot (\mathbf{g} - \mathbf{x}_j(t))]$$

где:

- χ — нормирующий множитель (сжимающий коэффициент);
- φ_r — коэффициент влияния личного лучшего решения (когнитивная составляющая);
- φ_s — коэффициент влияния глобального лучшего решения (социальная составляющая);
- r_p, r_g — случайные числа, равномерно распределённые в интервале $(0,1)$;
- $\mathbf{p}_j$ — личное лучшее положение частицы j ;
- $\mathbf{g}$ — глобальное лучшее положение всего роя.

Сжимающий коэффициент χ вычисляется как:

$$\chi = \frac{2}{|2 - \varphi - \sqrt{\varphi^2 - 4\varphi}|}, \quad \varphi = \varphi_r + \varphi_s$$

При этом обязательно выполняется условие:

$$\varphi > 4$$

Это условие гарантирует, что траектории частиц будут сходиться, а не расходиться. Наиболее часто используются значения $\varphi_r = 2.05$, $\varphi_s = 2.05$, откуда $\varphi = 4.1$, $\chi \approx 0.7298$.

После вычисления новой скорости положение частицы корректируется:

$$\mathbf{x}_j(t+1) = \mathbf{x}_j(t) + \mathbf{v}_j(t+1)$$

В качестве критерия останова используется достижение фиксированного количества итераций.

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки rpyqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

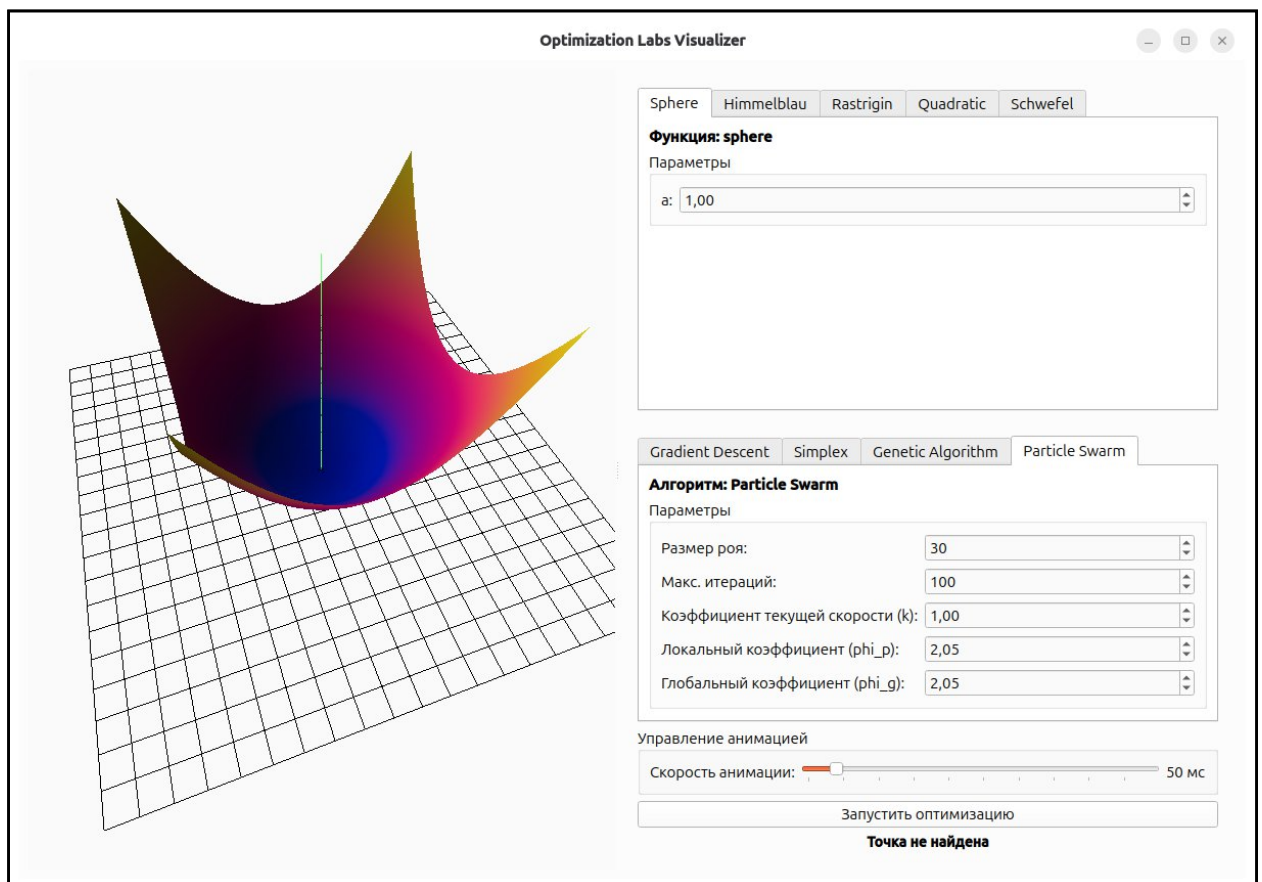


Рисунок 1 — Главное окно приложения

2. Доступные функции: Sphere, Himmelblau, Rastrigin, Quadratic и Schwefel (рисунок 2).

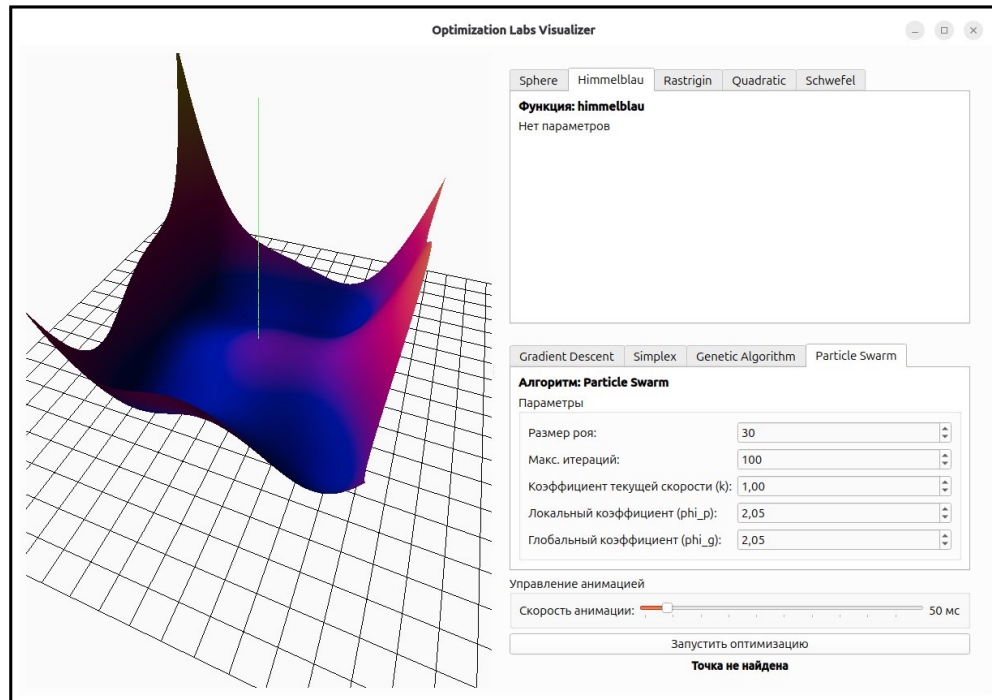


Рисунок 2 — Доступные функции

3. Доступные алгоритмы оптимизации: градиентный спуск с постоянным шагом, симплекс-метод, генетический алгоритм и алгоритм роя частиц (рисунок 3).

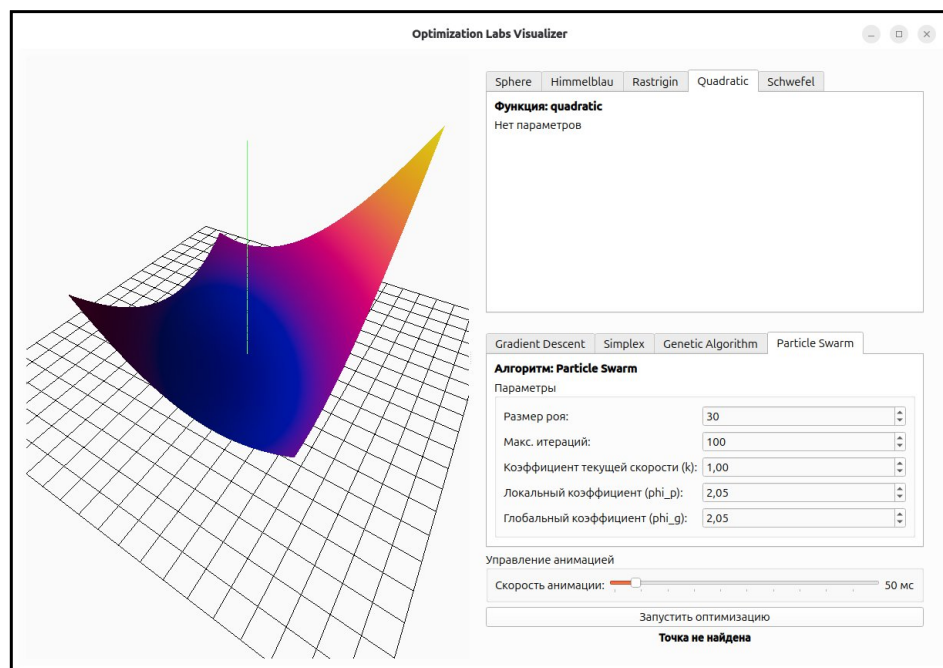


Рисунок 3 — Доступные алгоритмы оптимизации

4. Запустим алгоритм поиска минимума функции Schwefel (рисунок 4).

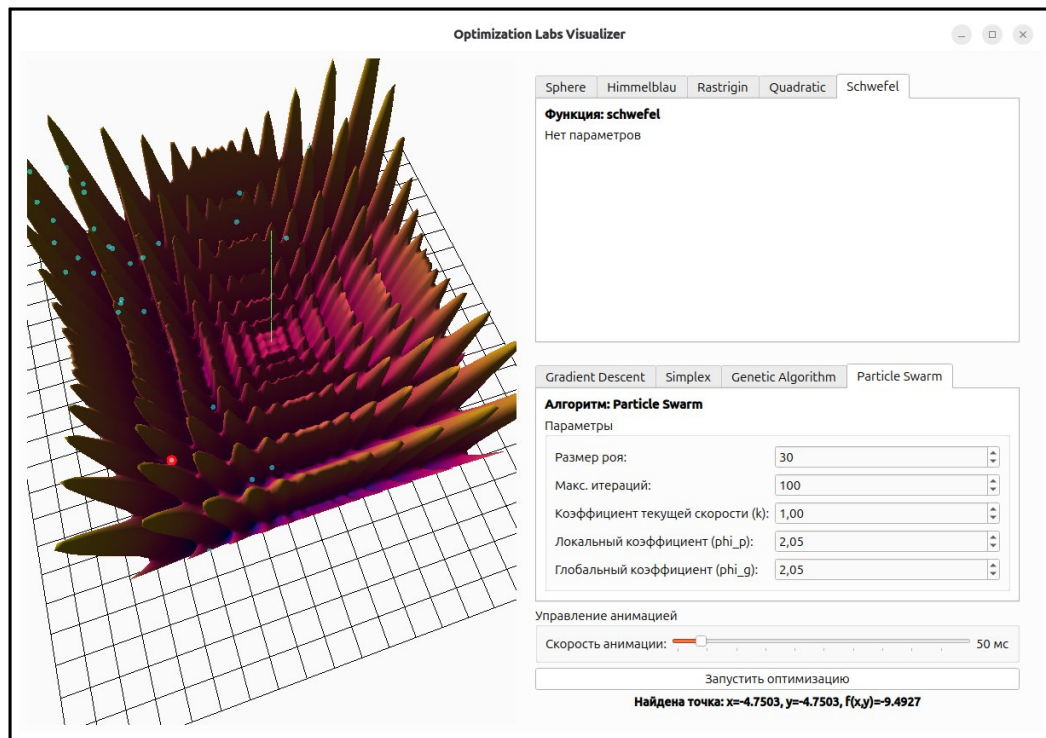


Рисунок 4 — Запуск алгоритма оптимизации

5. Запустим алгоритм поиска минимума функции Rastrigin (рисунок 5).

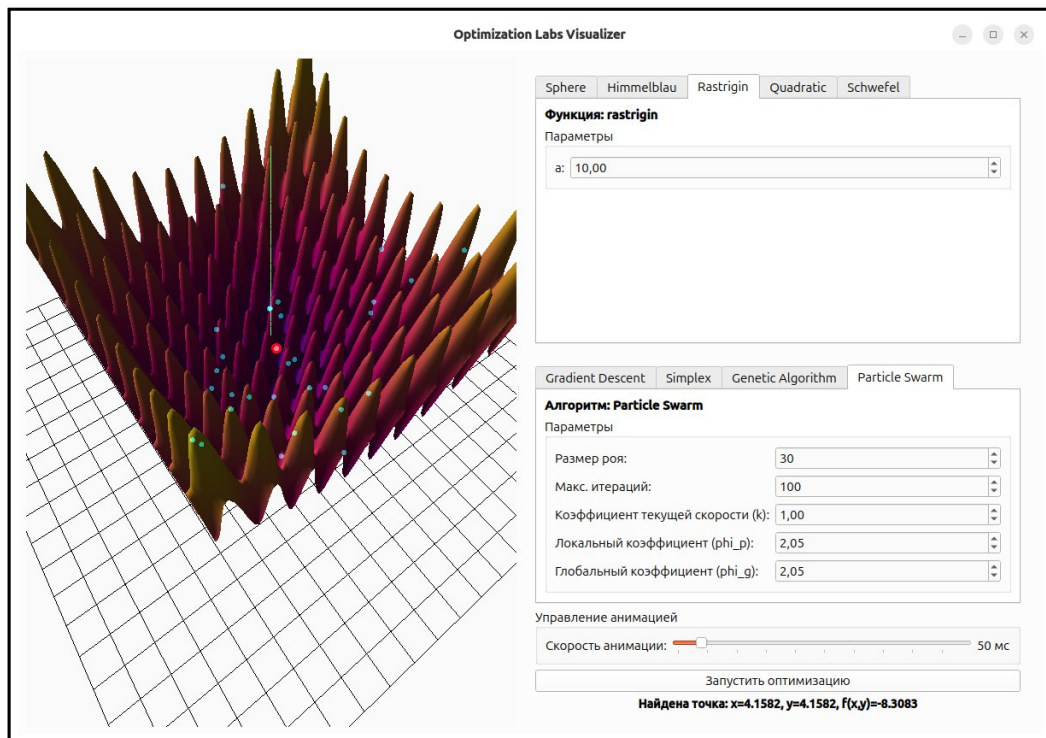


Рисунок 5 — Запуск алгоритма

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение алгоритма роя частиц для решения задачи поиска минимума функции.

Листинг программы.

```
import numpy as np

class Particle:
    def __init__(self, swarm):
        self.current_position = self._get_init_position(swarm)
        self.local_best_position = self.current_position.copy()
        self.local_best_final_func = swarm.get_final_func(self.current_position)
        self.velocity = self._get_init_velocity(swarm)

    def _get_init_position(self, swarm):
        return np.random.rand(swarm.dimension) * (swarm.maxvalues - swarm.minvalues) + swarm.minvalues

    def _get_init_velocity(self, swarm):
        minval = -(swarm.maxvalues - swarm.minvalues)
        maxval = swarm.maxvalues - swarm.minvalues
        return np.random.rand(swarm.dimension) * (maxval - minval) + minval

    def next_iteration(self, swarm):
        rnd_current = np.random.rand(swarm.dimension)
        rnd_global = np.random.rand(swarm.dimension)

        velo_ratio = swarm.local_velocity_ratio + swarm.global_velocity_ratio
        denom = np.abs(2.0 - velo_ratio - np.sqrt(velo_ratio ** 2 - 4.0 * velo_ratio))
        common_ratio = (2.0 * swarm.current_velocity_ratio) / denom if denom > 1e-10 else 1.0

        new_v1 = common_ratio * self.velocity
        new_v2 = common_ratio * swarm.local_velocity_ratio * rnd_current * \
            (self.local_best_position - self.current_position)
        new_v3 = common_ratio * swarm.global_velocity_ratio * rnd_global * \
            (swarm.global_best_position - self.current_position)

        self.velocity = new_v1 + new_v2 + new_v3
        self.current_position += self.velocity

        # Зажимаем в границы области поиска
        self.current_position = np.clip(self.current_position, swarm.minvalues, swarm.maxvalues)

        final_func = swarm.get_final_func(self.current_position)
        if final_func < self.local_best_final_func:
            self.local_best_position = self.current_position.copy()
            self.local_best_final_func = final_func

class Swarm:
    def __init__(self, swarmsize, minvalues, maxvalues,
                 current_velocity_ratio, local_velocity_ratio, global_velocity_ratio,
                 func, func_params):
        self.dimension = len(minvalues)
        self.swarmsize = swarmsize
        self.minvalues = np.array(minvalues)
        self.maxvalues = np.array(maxvalues)
        self.current_velocity_ratio = current_velocity_ratio
        self.local_velocity_ratio = local_velocity_ratio
```

```

self.global_velocity_ratio = global_velocity_ratio
self.global_best_final_func = None
self.global_best_position = None
self.func = func
self.func_params = func_params
self.swarm_particles = [Particle(self) for _ in range(swarmsize)]

def get_final_func(self, position):
    final_func = self.func(position[0], position[1], **self.func_params)
    if self.global_best_final_func is None or final_func < self.global_best_final_func:
        self.global_best_final_func = final_func
        self.global_best_position = position.copy()
    return final_func

def next_iteration(self):
    for particle in self.swarm_particles:
        particle.next_iteration(self)

def get_positions(self):
    return [(p.current_position[0], p.current_position[1]) for p in self.swarm_particles]

def particle_swarm(func, x_range, y_range, num_particles=30, generations=100,
    current_velocity_ratio=1.0, local_velocity_ratio=2.05,
    global_velocity_ratio=2.05, clear_view=None, **func_params):
    """Генератор для анимации"""
    minv = [x_range[0], y_range[0]]
    maxv = [x_range[1], y_range[1]]
    swarm = Swarm(num_particles, minv, maxv,
        current_velocity_ratio, local_velocity_ratio, global_velocity_ratio,
        func, func_params)

    # Начальное положение роя
    yield sorted(swarm.get_positions(), key=lambda p: func(*p, **func_params), reverse=True)

    for _ in range(generations):
        swarm.next_iteration()
        if clear_view:
            clear_view()
        yield sorted(swarm.get_positions(), key=lambda p: func(*p, **func_params), reverse=True)

```